Transport Coverage Report

Every operation that exists on at least one transport, mapped against the other two, with destination-reachability flagged.

Sources: USB_Command_Handler.c, USB_Command_Adapter.c (per project), RS485_Protocol.h, RS485_Command_Adapter.c, RS485_Task.c, Clean_Scheduler.c, CYW43_Web_Server.c.

Legend

Tag	Meaning
ok	Reachable on every transport that should carry it.
warn	Reachable, but only via the DEBUG_CMD CLI tunnel — no native opcode / no native route.
miss	Operation exists on at least one transport but is missing on another that needs it.
drop	Command IS sent on a transport but the destination has no handler — silently dropped.

RS-485 opcodes with no slave handler (silent drop)

These opcodes are defined in RS485_Protocol.h and may be sent by the CCH master, but the slave-side RS485_Command_Adapter does not implement them. The bytes go on the wire and nothing happens at the destination.

- PING (0x01)
- RESET (0x03)
- SET_ADDRESS (0x05)
- SYNC_TIME (0x06)
- DISCOVER (0x07) / DISCOVER_RESPONSE (0x08)
- TRIGGER_CLEAN (0x09)
- BUSY (0x12)
- POLL_EVENTS (0x22) / EVENT_RESPONSE (0x23)
- HEARTBEAT (0x24)
- TRANSACTION (0x30) / TRANSACTION_BATCH (0x31) / TRANSACTION_ACK (0x32) / GET_TRANSACTIONS (0x33)
- CARD_DETECTED (0x40) / CARD_DATA (0x41) / CARD_UPDATE (0x42) / CARD_BLACKLIST (0x43)
- SET_CONFIG (0x51) / CONFIG_RESPONSE (0x52)
- GET_CONFIG (0x50) — handler is a TODO that returns no payload

Web / HTTP endpoints (CCH)

Method	Path	Handler / behaviour
GET	/api/status	cgi_api_status — JSON snapshot: slave table, module states, RTC, SD.
POST	/api/control	cgi_api_control — start/stop LOCAL CCH modules only (toggle).
POST	/api/cmd	cgi_api_cmd — addr=0 runs USB_Command_HandleString locally; addr=1..N tunnels via RS485_CMD_DEBUG_CMD (universal CLI bridge).
GET	/, /<path>	fs_open_custom — static files from SD card /www/.

Operation × Transport matrix

Each row is one logical operation. The colour chip on the left encodes the destination-reachability tag (see Legend). Cells marked “—” mean the operation has no surface on that transport.

Operation	USB CLI	RS-485	Web / HTTP	Note
Ping device	—	PING (0x01)	—	Opcode defined; no slave handler.
Reset device	sys reboot (local)	RESET (0x03)	—	RS-485 RESET has no slave handler. USB only resets the local device.
Get device info	sys version	GET_INFO (0x04)	via /api/cmd	RS-485 returns FW/serial; web has no dedicated endpoint.
Set slave address	—	SET_ADDRESS (0x05)	—	Opcode defined; no slave handler.
Sync RTC to slave	—	SYNC_TIME (0x06)	—	Master sends SYNC_TIME (RS485_Task.c); slaves do not handle it. Use `remote <addr> rtc set` as workaround.
Discover slaves	—	DISCOVER (0x07) / RSP (0x08)	—	Opcodes defined; no slave handler. CCH cannot auto-enumerate the bus.
Heartbeat	sys tasks (local)	HEARTBEAT (0x24)	—	Opcode defined; no slave handler.
Busy notify	—	BUSY (0x12)	—	Opcode defined; no slave handler.
ACK / NAK	—	ACK (0x10) / NAK (0x11)	—	Implemented on both sides.
Poll status	sys status (local)	POLL_STATUS (0x20) / RSP (0x21)	/api/status	Master polls slaves; web aggregates.
Poll events	—	POLL_EVENTS (0x22) / RSP (0x23)	—	Opcodes defined; no slave handler.
Start dispense	dispenser start [L]	DEBUG_CMD tunnel	/api/cmd addr=N cmd='dispenser start'	No native DISPENSE_START opcode; routed through CLI bridge.
Stop dispense	dispenser stop	DEBUG_CMD tunnel	/api/cmd cmd='dispenser stop'	Same as above.

Operation	USB CLI	RS-485	Web / HTTP	Note
Topup card	dispenser topup <L> / card topup	DEBUG_CMD tunnel	/api/cmd cmd='dispenser topup N'	No native CARD_UPDATE / topup opcode handler on slave.
Read balance	sys status, card log	POLL_STATUS payload	/api/status	Reachable everywhere.
Trigger clean	clean start [vol] [sec] (MyWota)	TRIGGER_CLEAN (0x09)	/api/cmd cmd='clean start ...'	Master queues TRIGGER_CLEAN (Clean_Scheduler.c) but slave RS485_Command_Adapter has no handler. Web works only because it falls back to DEBUG_CMD.
Abort clean	clean stop (MyWota)	—	/api/cmd cmd='clean stop'	No native abort opcode; web works via DEBUG_CMD only.
Clean status	clean status (MyWota)	—	/api/cmd cmd='clean status'	Master-side `clean status` is local; per-slave clean state must be tunneled.
Start wash	washstart [s] [opt]	DEBUG_CMD tunnel	/api/cmd cmd='washstart'	BigYellow uses flat top-level command; no native opcode.
Stop wash	washstop	DEBUG_CMD tunnel	/api/cmd cmd='washstop'	Same as above.
Format / init card	card format	—	—	USB only. Cannot remotely re-init a slave's card session.
Card log	card log <UID>	—	—	USB only.
Card recover	card recover <UID>	—	—	USB only.
Card decrypt	card decrypt	—	—	USB only.
Card transactions push	—	TRANSACTION (0x30) / BATCH (0x31) / GET_TX (0x33)	—	Whole transaction-sync family defined; no slave handlers. Currently substituted by DEBUG_CMD log capture.
Card detected event	—	CARD_DETECTED (0x40)	—	Opcode defined; no slave handler.

<table>
<tr><th>Operation</th><th>USB CLI</th><th>RS-485</th><th>Web / HTTP</th><th>Note</th></tr>
<tr><td>Card data transfer</td><td>—</td><td>CARD_DATA (0x41)</td><td>—</td><td>Opcode defined; no slave handler.</td></tr>
<tr><td>Card update (balance)</td><td>dispenser topup</td><td>CARD_UPDATE (0x42)</td><td>via /api/cmd</td><td>Opcode defined; no slave handler.</td></tr>
<tr><td>Card blacklist</td><td>—</td><td>CARD_BLACKLIST (0x43)</td><td>—</td><td>Opcode defined; no slave handler.</td></tr>
<tr><td>WiFi scan</td><td>wifi scan</td><td>—</td><td>—</td><td>USB only on CCH.</td></tr>
<tr><td>WiFi connect</td><td>wifi connect <ID|pwd></td><td>—</td><td>—</td><td>USB only. Cannot provision a slave's WiFi (slaves don't have WiFi anyway).</td></tr>
<tr><td>WiFi disconnect / saved / forget / reinit</td><td>wifi disconnect|saved|forget|reinit</td><td>—</td><td>—</td><td>USB only.</td></tr>
<tr><td>WiFi status</td><td>wifi status</td><td>—</td><td>/api/status (partial)</td><td>/api/status reflects connectivity but not network state machine.</td></tr>
<tr><td>RTC read</td><td>rtc status</td><td>—</td><td>/api/status (current_time)</td><td>Reachable on USB and web; RS-485 lacks a dedicated read opcode.</td></tr>
<tr><td>RTC set</td><td>rtc set <date> <time></td><td>—</td><td>—</td><td>USB only. To set slave RTC, must use `remote <addr> rtc set`.</td></tr>
<tr><td>Get config (slave)</td><td>sys config (local)</td><td>GET_CONFIG (0x50)</td><td>via /api/cmd 'sys config'</td><td>Slave RS485_Command_Adapter::cmd_get_config is a TODO and returns no data.</td></tr>
<tr><td>Set config (slave)</td><td>sys set <param> <val> (local)</td><td>SET_CONFIG (0x51)</td><td>via /api/cmd 'sys set ...'</td><td>Opcode defined; no slave handler.</td></tr>
<tr><td>Config response</td><td>—</td><td>CONFIG_RESPONSE (0x52)</td><td>—</td><td>Counterpart to GET_CONFIG; never emitted.</td></tr>
<tr><td>Factory reset / save / defaults-apply</td><td>sys factory-reset|factory-save|defaults-apply</td><td>—</td><td>—</td><td>USB only. No remote way to factory-reset a slave.</td></tr>
<tr><td>Module list</td><td>mod list</td><td>—</td><td>/api/status</td><td>Web exposes module state read-only. RS-485 has no module-control opcode.</td></tr>
</table>

Operation	USB CLI	RS-485	Web / HTTP	Note
Module start / stop	mod start <m> / mod stop <m>	—	/api/control	/api/control toggles LOCAL CCH modules only. To start/stop a slave's module, must use `/api/cmd remote ...`.
List SD files	fs ls	—	GET /<file> (read-only static)	Web serves static files; no listing endpoint. RS-485 has nothing.
Read SD file	fs cat <name>	—	GET /<path>	Web reads /www/* from SD; CLI reads any file. No RS-485 path.
Find by UID	fs find <UID>	—	—	USB only.
Format / wipe SD	fs format	—	—	USB only.
Mount / unmount MSC	fs mount / fs unmount	—	—	USB only (physically requires USB anyway).
Re-init SD	sdinit (CCH)	—	—	USB only.
List RS-485 slaves	rs485	(implicit polling)	/api/status	Master keeps slave table; web exposes JSON.
Forward CLI to slave	remote <addr> <cmd>	DEBUG_CMD (0xF1) + DEBUG_FETCH (0xF2)	/api/cmd addr=N	Universal escape hatch — every transport supports it.
Debug log channel	(USB log stream)	DEBUG_LOG (0xF0)	—	RS-485 carries slave debug logs to master; web has no log-stream endpoint.
Start web server	webserver start	—	(self)	Local-only command; bootstraps the web transport.
Web server status	webserver status	—	/api/status (partial)	
Filter status / reset / set	filter status|reset|set <ml> (CCH)	—	—	USB only on CCH.
Throughput stats	stats (CCH)	—	—	USB only on CCH.
Fault status / clear	fault status|clear (MyWota)	—	—	USB only on MyWota. CCH cannot read or clear a slave fault remotely.

Operation	USB CLI	RS-485	Web / HTTP	Note
WDT log read / save / erase	`sys wdt [save|erase|raw]`	—	—	USB only.
Tasks / stack high-water	`sys tasks`	—	—	USB only.
BOOTSEL reflash	`sys bootloader`	—	—	USB only (intentional — physical re-flash path).
FW update start	—	`FW_START (0x60)`	—	Master sends; handled inside RS485_Task on slave (not yet via adapter).
FW data chunk	—	`FW_DATA (0x61)`	—	Same as FW_START.
FW verify	—	`FW_VERIFY (0x62)`	—	Same as FW_START.
FW apply	—	`FW_APPLY (0x63)`	—	Same as FW_START.
Security mode toggle	`sys security <on|off>`	—	—	USB only. Cannot toggle on a slave without `remote ...`.

Where commands fail to reach their destination

Tag	Count	Meaning
ok	8	Reachable on every transport that should carry it
warn	14	Reachable only via DEBUG_CMD CLI tunnel (no native opcode/route)
miss	24	Operation exists on USB but never plumbed to RS-485 or web
drop	17	RS-485 opcode SENT but slave has no handler — silently dropped

Top action items

• Implement TRIGGER_CLEAN (0x09) on the slave

CCH master already queues this opcode in Clean_Scheduler.c; slave currently ignores it. The web `/api/cmd` workaround masks the bug.

• Implement SYNC_TIME (0x06) on the slave

RS485_Task on CCH sends SYNC_TIME but slaves do not accept it. RTC drift between master and slaves is currently only fixable via `remote rtc set`.

• Finish GET_CONFIG / SET_CONFIG (0x50 / 0x51)

cmd_get_config in the slave adapter is a TODO. There is no way to read or push a slave's config without a CLI tunnel.

• Add transaction-sync handlers (0x30–0x33)

Whole opcode family unused. Today the only way the master sees slave transactions is by parsing log output captured through DEBUG_CMD.

• Add module-control opcodes (or a generic remote `mod start/stop`)

/api/control only toggles LOCAL CCH modules. A slave's modules can only be stopped via DEBUG_CMD.

• Define MISSING vs. WAIT-FOR-CARD semantics for `card *` opcodes

card format/log/recover/decrypt have no remote pathway. If field service ever needs them, they currently require physical USB.

• Consider deprecating DEBUG_CMD as the universal bridge

Roughly half of the “working remote” operations only work because they fall back to a textual CLI parse on the slave — which defeats the structured protocol.